

ClauseLens

AI-Powered Legal Contract Intelligence

ClauseLens is an AI-assisted contract-analysis platform that supports the first-pass review of legal agreements. Upload a PDF or Word contract and ClauseLens identifies its clauses, extracts the key terms (parties, dates, financial figures, governing law), evaluates potential risk, and presents the findings through a clean, interactive interface - turning hours of manual reading into a structured review that takes minutes.

It is built to be **private, explainable, and deployable**: the analysis runs on a lightweight, CPU-only machine-learning pipeline with no third-party AI calls, so contract text never leaves the application at analysis time.

1. Key features

- **AI-assisted clause identification** - classifies each contract section into one of 41 clause types.
- **Legal document processing** - reads both PDF and Word (DOCX) contracts.
- **Key term extraction** - parties, effective date, term, governing law, monetary amounts, notice periods.
- **Contract risk assessment** - a transparent 0-100 risk score with ranked, evidence-cited findings.
- **Structured analysis reports** - a branded, exportable HTML report plus JSON data.
- **Contract library** - save analyses and browse portfolio-level analytics.
- **Clause search** - find similar clauses across the whole library.
- **Contract knowledge center** - flashcards and articles that teach the underlying concepts.
- **Interactive dashboard** - live metrics and visualizations of your reviewed contracts.

2. System workflow

```
Contract Upload
|
Document Processing (PDF / DOCX)
|
Text Extraction & Cleaning
|
NLP / AI Analysis
|
Clause Identification (41 types)
|
Key Term Extraction
|
Risk Assessment (0-100)
|
Report Generation
|
Dashboard / Library / Search
```

3. Technology stack

Technology	Purpose
Python 3.13	Core language for the pipeline and app

Streamlit	Interactive web UI and deployment
scikit-learn	Clause classifier (TF-IDF + Logistic Regression)
CUAD dataset	Expert-labelled training data (41 clause types)
pdfplumber	PDF text extraction
python-docx / docx2txt	DOCX text extraction
pandas / NumPy / SciPy	Data handling and numerics
Plotly	Interactive charts (risk gauge, distributions)
SQLite	Contract library storage
pytest	Automated testing (73 tests)

(No large language model or transformer is used - see Sec. 4.)

4. AI / ML approach

- **Model:** a scikit-learn pipeline - a union of **word (1-2 gram) and character (3-5 gram) TF-IDF** features feeding a class-balanced **Logistic Regression**, selected over a calibrated Linear SVC on validation macro-F1.
- **Dataset:** the **Contract Understanding Atticus Dataset (CUAD)** - 41 expert-annotated clause categories; trained on 10,901 clauses, evaluated on 1,368.
- **Clause recognition:** each section is classified with a confidence threshold, so low-confidence predictions abstain rather than mislabel; a keyword baseline provides a transparent fallback.
- **Information extraction:** rule-based (regex/heuristics) for parties, dates, money, governing law, term, and notice periods - precise and fully explainable.
- **Risk analysis:** a rule-based engine combining risky clauses present, protective clauses missing, and red-flag language into a 0-100 score with evidence.

Why classical ML instead of an LLM? A few-MB, CPU-only model is private, instant, explainable, deterministic, free to run, and cannot hallucinate a clause - the properties a legal tool needs. A transformer could edge accuracy higher, at ~100x the size and cost; that trade was deliberately not taken for a deployable, private tool.

5. User experience

Pages: Dashboard · Analyze Contract · Contract Library · Find Clauses · Learn · About.

Design philosophy:

- A professional legal-tech interface with a deep-navy / teal / gold palette.
- Clear information hierarchy and risk-focused visualization (green = low, gold/amber = medium, red = high).
- Serif display typography (Playfair Display) for headings; a modern sans-serif (Manrope) for body.
- Subtle, purposeful animations (fade-in, hover elevation, staged processing, flip flashcards).
- Responsive layout and reduced-motion support.

6. Results

For a given contract, ClauseLens produces:

- an **overall risk score** (0-100) and level (Low / Medium / High);
- **clause categories** with confidence;
- **extracted key terms** (parties, dates, money, governing law, notice);
- ranked **risk findings** with evidence and recommendations;
- a plain-English **executive summary**; and
- a **downloadable report** (HTML) and data export (JSON).

Model evaluation (held-out CUAD test set):

Metric	Value
Accuracy	77.1%
Macro-F1	0.70
Weighted-F1	0.75
Baseline (majority/keyword) accuracy	16.2%
Improvement over baseline	~4.8x
Clause types	41
Training / test clauses	10,901 / 1,368

Accuracy reflects a lightweight, deployable model; it is a first-pass triage aid, not a substitute for legal judgement.

7. Project architecture

```

User
|

Streamlit UI (app.py)
|

Document layer -- Preprocessing -- Extraction -- Analysis -- core.py
(pdf/docx) (clean text) (clauses+ML, (risk engine, (AnalysisResult)
entities) summarizer) |

|-----|

Report (HTML) SQLite library TF-IDF search

```

Each layer is a small, independently tested Python package under `src/`, orchestrated by `core.py`.

8. Why this project?

Legal contracts hold critical information that is time-consuming and error-prone to review by hand. ClauseLens explores how NLP and classical machine learning can structure that first-pass review - identifying clauses, surfacing risk, and making contracts searchable - while remaining private, explainable, and cheap enough to deploy for free.

9. Limitations

ClauseLens is a **decision-support tool** and **does not replace qualified legal advice**.

- Model accuracy is ~77% on an imbalanced 41-class task; unusual or ambiguous contracts may classify poorly.
- There is no OCR, so scanned image-only PDFs are flagged rather than processed.
- The risk score is a transparent heuristic, not a legal judgement.
- The library is single-tenant (no authentication yet).

All results should be reviewed by a qualified professional.

10. Future roadmap

- **Phase 1 (current):** contract analysis - clauses, terms, risk, reports.
- **Phase 2:** semantic clause search across the library. *(implemented)*
- **Phase 3:** playbook / contract comparison.
- **Phase 4:** version comparison and redlining.
- **Phase 5:** OCR for scanned documents.
- **Phase 6:** authentication and organization/workspace support.
- **Phase 7:** API architecture and scalable, multi-tenant deployment.

11. Project structure

```
legal-contract-analyzer/  
|-- app.py # Streamlit UI  
|-- src/  
| |-- core.py # pipeline orchestrator  
| |-- report.py # branded HTML/JSON report  
| |-- security.py # escaping, upload validation  
| |-- document/ # pdf_parser, docx_parser  
| |-- preprocessing/ # text_cleaner  
| |-- extraction/ # clause_extractor, entity_extractor, ml_classifier  
| |-- analysis/ # risk_engine, summarizer  
| |-- search/ # semantic_search (TF-IDF)  
| |-- storage/ # db (SQLite)  
| |-- content/ # education (flashcards + articles)  
|-- training/ # CUAD prep + classifier training  
|-- models/ # committed clause_classifier.joblib  
|-- tests/ # 73 tests  
|-- sample_contracts/ # demo contract  
|-- docs/ # documentation + this case study  
|-- requirements.txt
```

12. Local setup

```
git clone <repository-url>  
cd legal-contract-analyzer  
python -m venv venv  
venv\Scripts\activate # Windows (source venv/bin/activate on macOS/Linux)  
pip install -r requirements.txt  
streamlit run app.py
```

Open <<http://localhost:8501>> and upload a contract (a sample is in `sample_contracts/`).

13. Screenshots

(Add images when publishing.)

- Dashboard - docs/screenshots/dashboard.png
- Analyze Contract - docs/screenshots/analyze.png
- Analysis results & risk - docs/screenshots/results.png
- Contract Library - docs/screenshots/library.png
- Find Clauses - docs/screenshots/search.png
- Learn - docs/screenshots/learn.png

14. Demo

Upload a contract -> watch the staged analysis -> review the extracted clauses and key terms -> inspect the 0-100 risk score and findings -> save it to your library -> search for a clause -> download the report.

15. Disclaimer

ClauseLens is an AI-assisted decision-support tool for contract review. It does not constitute legal advice and does not replace a qualified attorney. All outputs should be verified by a legal professional.

16. Author

Podugu Bala Veera Venkata Sunil

- GitHub: *(add your profile URL)*
- LinkedIn: *(add your profile URL)*
- Project: *(add your repository / live demo URL)*

17. LinkedIn & GitHub copy

LinkedIn headline

Built ClauseLens - an AI-powered legal contract analyzer (Python · scikit-learn · Streamlit) that turns hours of contract review into a minutes-long, structured first pass.

LinkedIn short version (~600 characters)

*I built **ClauseLens**, an AI-powered legal contract analyzer. Upload a PDF or Word contract and it identifies clauses (41 types), extracts key terms, scores risk 0-100, and generates a structured report - turning hours of review into minutes. I used classical ML (TF-IDF + Logistic Regression on the CUAD dataset, ~77% accuracy) rather than an LLM, so it's private, explainable, and deploys for free. Built with Python, scikit-learn and Streamlit. It's a decision-support tool, not a replacement for a lawyer.*

#MachineLearning #LegalTech #Python #NLP

LinkedIn detailed version (~1,300 characters)

ClauseLens - an AI-powered legal contract analyzer

>

Reviewing contracts is slow and repetitive: you read pages of boilerplate to find the few clauses that carry risk. I built ClauseLens to automate that first pass.

>

Upload a PDF or Word contract and it:

- * identifies each clause (41 types) with a model trained on the CUAD dataset*
- * extracts parties, dates, financial terms and governing law*
- * scores risk 0-100 - flagging risky clauses AND missing protections, each finding citing its evidence*
- * writes a plain-English summary and a shareable report*
- * makes every clause semantically searchable*

>

*The key engineering decision was to use **classical ML (TF-IDF + Logistic Regression)**, not an LLM. The result reaches ~77% accuracy on 41 classes, runs on CPU in milliseconds, keeps documents private, is fully explainable, and deploys for free - properties that matter more than a few points of accuracy for a legal tool.*

>

Built with Python, scikit-learn, and Streamlit; hardened against the OWASP Web & LLM Top 10 and covered by 73 automated tests.

>

! It's a decision-support tool, not a replacement for a qualified attorney.

>

#MachineLearning #LegalTech #Python #DataScience #NLP #scikitlearn #Streamlit #AI

GitHub repository tagline

*AI-powered legal contract analyzer - clause classification, key-term extraction, risk scoring, and search.
Python · scikit-learn · Streamlit.*

GitHub topics

`legal-tech · nlp · machine-learning · contract-analysis · scikit-learn · streamlit · python · text-classification · document-intelligence · cuad · tfidf · data-science`